

ECE 786: Final Project - Ashwin Mathew Olakangal

Task 1:

Using **aggregated IPC** values for BFS and LUD

Benchmark Name	kernel_name	kernel_launch_uid	IPC without cache bypassing	IPC with cache bypassing	Percent change in IPC	Benchmark category
BFS	Aggregate of all 9 kernels	1-9	24.2689	40.3648	+66.32%	Cache Unfriendly
LPS	_Z13GPU_laplace3diiiPfS_	1	371.5598	545.6426	+46.85%	Cache Unfriendly
NQU	_Z24solve_nqueen_cuda_kerneliiPjS_S_S_i	1	34.2939	34.3879	+0.27%	Cache Insensitive
LUD	Aggregate of all 46 kernels	1-46	41.8475	43.5609	+4.09%	Cache Unfriendly
BP	_Z22bpnn_layerforward_CUDA PfS_S_S_i	1	665.0260	712.9353	+7.20%	**Cache Unfriendly
BP	_Z24bpnn_adjust_weights_cudaPfS_iS_S_	2	129.7257	126.9964	-2.10%	**Cache friendly
HS	_Z14calculate_tempiPfS_S_iiiiffff	1	356.3531	366.1858	+2.76%	Cache Insensitive

Since one kernel in BP is cache friendly and other is cache unfriendly, I'm taking the aggregate of BP which is 553.0369 without bypass and 581.3123 with bypass resulting in **overall benchmark behaviour of being cache unfriendly (+5.11%)

Task 2:

Implementation details:

For each global memory access, I take the address, convert it to an L1 cache block address, and update a counter using the format of SM id, kernel launch uid, and block address. At the end of the first run, these counters are dumped into a profile file so they can be reused later.

In the second run, I load the profile file before simulation starts and use those saved counters to guide cache bypassing. The bypass decision is made in the load/store path. For each global memory request, I check the profiled reference count of its corresponding L1 data block for that SM and kernel. If the reference count is less than 3, I bypass the L1 data cache and send the request directly through the bypass path.

I modified *shader.cc* to add the actual profiling and profile-based bypass logic in the load/store path, and to make sure the same bypass decision is handled correctly again on the return/fill path. I also modified *shader.h* to add the new config options for profiling and profile-based bypass so the simulator can recognize the new flags and threshold cleanly. In *gpu-sim.cc* and *gpu-sim.h* I made changes to store the per-SM, per-kernel block reference profiles, dump them to a file after run 1, and load them back before run 2.

Using **aggregated IPC** values for BFS and LUD

Benchmark Name	kernel_name	kernel_launch_uid	IPC without cache bypassing	IPC with profile based cache bypassing	Percent change in IPC	Benchmark category
BFS	Aggregate of all 9 kernels	1-9	24.2689	28.9053	+19.10	Cache Unfriendly
LPS	_Z13GPU_laplace3diiiPfS_	1	371.5598	530.5136	+42.78%	Cache Unfriendly
LUD	Aggregate of all 46 kernels	1-46	41.8475	42.9064	+2.53%	Cache Insensitive
BP	_Z22bpnn_layerforward_CUDA_PfS_S_S_i	1	665.0260	729.8881	+9.75%	**Cache Unfriendly
BP	_Z24bpnn_adjust_w	2	129.7257	133.2012	+2.68%	**Cache Insensitive

	eights_cud aPfiS_iS_ S_					
--	-------------------------------	--	--	--	--	--

****Since one kernel in BP is cache unfriendly and other is cache insensitive, I'm taking the aggregate of BP which is 553.0369 without bypass and 598.3546 with bypass resulting in overall benchmark behaviour of being cache unfriendly (+8.19%)**

Running with profile based cache bypassing:

1. Copy all the files that were modified: namely *shader.cc/shader.h* and *gpu-sim.cc/gpu-sim.h*

2. Modify the gpgpusim.config file:

For first run to generate profiling data:

```
gpgpu_generate_L1D_profile 1
-gpgpu_use_L1D_profile_bypass 0
-gpgpu_gmem_skip_L1D 0
-gpgpu_L1D_profile_outfile /accel-sim/accel-sim-framework/task2/l1d_profile.txt
-gpgpu_L1D_profile_threshold 3

#-gpgpu_generate_L1D_profile 0
#-gpgpu_use_L1D_profile_bypass 1
#-gpgpu_gmem_skip_L1D 0
#-gpgpu_L1D_profile_infile /accel-sim/accel-sim-framework/task2/l1d_profile.txt
#-gpgpu_L1D_profile_threshold 3
```

Create relevant folder called task2 at /accel-sim/accel-sim-framework/task2

Make and run the relevant trace that you want from /accel-sim/accel-sim-framework. The profiling data for that trace will be generated inside task2 folder called l1d_profile.txt

For second run after the profiling data has been generated:

```
#-gpgpu_generate_L1D_profile 1
#-gpgpu_use_L1D_profile_bypass 0
#-gpgpu_gmem_skip_L1D 0
#-gpgpu_L1D_profile_outfile /accel-sim/accel-sim-framework/task2/l1d_profile.txt
#-gpgpu_L1D_profile_threshold 3

gpgpu_generate_L1D_profile 0
-gpgpu_use_L1D_profile_bypass 1
-gpgpu_gmem_skip_L1D 0
-gpgpu_L1D_profile_infile /accel-sim/accel-sim-framework/task2/l1d_profile.txt
-gpgpu_L1D_profile_threshold 3
```

Make and run the relevant trace that you want from /accel-sim/accel-sim-framework. Ensure the profiling data for that trace is inside task2 folder called l1d_profile.txt